

LAPORAN PRATIKUM STRUKTUR DATA

PEKAN 3

disusun Oleh:

Habib Al Faruq

2511532010

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum struktur data mengenai penggunaan Linked List di Java. Penulisan laporan ini bertujuan untuk membuat sebuah laporan tentang materi pemrograman Linked List di java.

Penyusunan laporan praktikum ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada dosen pengampu mata kuliah praktikum pemrograman algoritma dan pemrograman, Bapak Dr. Wahyudi. S.T.M.T dan asisten praktikum, Uda Muhammad Galid Avero yang telah memberikan arahan dan masukan dalam proses penyusunan laporan praktikum ini. Penulis berharap laporan praktikum ini dapat memberikan manfaat bagi pembaca dalam mempelajari konsep penggunaan Linked List di Java

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Praktikum	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	2
 BAB II PEMBAHASAN.....	 3
2.1 QueryArray_2511532010 dan QueryArrayDriver_2511532010	3
2.2 QueueLinkedList_2511532010.....	4
2.3 ReverseData_2511532010	7
2.4 IterasiQueue_2511532010.....	9
 BAB III KESIMPULAN.....	 12
3.1 Ringkasan Hasil Praktikum.....	12
DAFTAR PUSTAKA.....	13
TUGAS	14

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Efisiensi sebuah perangkat lunak sangat bergantung pada pemilihan struktur data yang tepat, di mana Singly Linked List hadir sebagai solusi krusial bagi manajemen memori dinamis yang tidak bisa diakomodasi oleh array statis. Sistem ini bekerja dengan menghubungkan rangkaian simpul melalui referensi atau pointer, sehingga memungkinkan alokasi elemen dilakukan secara jauh lebih fleksibel. Dengan memahami mekanisme simpul dalam menyimpan data sekaligus alamat elemen berikutnya, seorang pengembang dapat merancang sistem penyimpanan yang mampu beradaptasi secara organik mengikuti dinamika kebutuhan aplikasi.

1.2 Tujuan Praktikum

Praktikum ini dirancang untuk membedah secara teknis operasi dasar pada Singly Linked List melalui bahasa Java, mulai dari pengenalan arsitektur simpul hingga logika manipulasi data yang lebih kompleks. Praktikan ditantang untuk menguasai teknik penyisipan dan penghapusan simpul di berbagai posisi secara presisi guna menjaga struktur data tetap akurat. Selain itu, aspek penelusuran serta pencarian menjadi fokus utama dalam sesi ini untuk memastikan bahwa integritas data tetap terjaga selama proses pengolahan berlangsung.

1.3 Manfaat Praktikum

Secara praktis, kegiatan ini sangat membantu dalam mempertajam logika pemrograman dan daya pikir abstrak mengenai manajemen data di balik sistem. Pemahaman yang matang terkait mekanisme pointer serta referensi objek memungkinkan praktikan untuk lebih cerdas dalam mengalokasikan sumber daya memori, terutama saat menangani arsitektur program yang rumit. Selain itu, keahlian

dalam mengelola simpul-simpul ini berfungsi sebagai batu pijakan utama untuk mempelajari struktur data yang lebih kompleks seperti Double Linked List, Stack, Queue, hingga model Tree dan Graph.

BAB II

PEMBAHASAN

2.1 NodeSLL_2511532010

```
1 package pekan5_2511532010;
2 import java.util.*;
3 public class NodeSLL_2511532010 {
4     int data_2010;
5     NodeSLL_2511532010 next_2010;
6     public NodeSLL_2511532010(int data_2010) {
7         this.data_2010 = data_2010;
8         this.next_2010 = null;
9     }
10 }
11
```

1. Deklarasi Package

Baris ini menunjukkan bahwa file kode ini berada dalam folder atau paket bernama pekan5_2511532010. Ini digunakan untuk mengorganisir file agar tidak bentrok dengan kelas lain.

2. Import Library

Baris ini berfungsi untuk mengimpor semua kelas dari paket java.util. Meskipun dalam potongan kode ini belum digunakan, library ini biasanya disiapkan jika nantinya kamu membutuhkan class seperti Scanner atau LinkedList bawaan Java.

3. Definisi Kelas

Ini adalah deklarasi kelas utama. Kelas ini bertindak sebagai "cetak biru" (blueprint) untuk objek node. Nama kelas ini mencantumkan NIM (2511532010), yang menunjukkan ini adalah tugas atau identitas spesifik.

4. Atribut Data

Variabel ini berfungsi untuk menyimpan nilai atau informasi utama di dalam node. Dalam kasus ini, node tersebut dirancang untuk menyimpan data bertipe bilangan bulat (integer).

5. Atribut Pointer/Reference

Ini adalah bagian krusial dari *Linked List*. Variabel ini bertipe objek dari kelas itu sendiri. Fungsinya adalah untuk menyimpan alamat (referensi) yang menunjuk ke node berikutnya dalam daftar. Jika nilainya null, berarti node tersebut adalah node terakhir.

6. Constructor

Ini adalah metode khusus yang dipanggil saat kamu membuat objek node baru (menggunakan kata kunci new). Constructor ini menerima satu parameter angka.

7. Inisialisasi Data

Di dalam constructor, nilai yang dimasukkan oleh pengguna akan disimpan ke dalam atribut `data_2010` milik node tersebut.

8. Inisialisasi Next

Saat pertama kali node dibuat, ia belum terhubung ke mana pun. Oleh karena itu, pointer `next_2010` secara otomatis diatur ke null sebagai penanda bahwa tidak ada node selanjutnya untuk saat ini.

2.2 HapusSLL_2511532010

```

HapusSLL_2511532010.java  NodeSLL_2511532010.java  PencarianSLL_2511532010.ja...  TambahSLL_25115
1 package pekan5_2511532010;
2
3 public class HapusSLL_2511532010 {
4
5     public static NodeSLL_2511532010 deteleHead(NodeSLL_2511532010 head_2010) {
6         if (head_2010 == null)
7             return null;
8         head_2010 = head_2010.next_2010;
9         return head_2010;
10    }
11
12    public static NodeSLL_2511532010 removeLastNode(NodeSLL_2511532010 head_2010) {
13        if (head_2010 == null || head_2010.next_2010 == null) {
14            return null;
15        }
16        NodeSLL_2511532010 secondLast_2010 = head_2010;
17        while (secondLast_2010.next_2010.next_2010 != null) {
18            secondLast_2010 = secondLast_2010.next_2010;
19        }
20        secondLast_2010.next_2010 = null;
21        return head_2010;
22    }
23
24    public static NodeSLL_2511532010 deleteNode(NodeSLL_2511532010 head_2010, int position_2010) {
25        NodeSLL_2511532010 temp_2010 = head_2010;
26        NodeSLL_2511532010 prev_2010 = null;
27
28        if (temp_2010 == null)
29            return head_2010;
30
31        if (position_2010 == 1) {
32            head_2010 = temp_2010.next_2010;
33            return head_2010;
34        }
35
36        for (int i = 1; temp_2010 != null && i < position_2010; i++) {
37            prev_2010 = temp_2010;
38            temp_2010 = temp_2010.next_2010;
39        }
40
41        if (temp_2010 != null) {
42            prev_2010.next_2010 = temp_2010.next_2010;
43        } else {
44            System.out.println("Data tidak ada");
45        }
46        return head_2010;
47    }
48
49    public static void printList(NodeSLL_2511532010 head_2010) {
50        if (head_2010 == null) {
51            System.out.println("List Kosong");
52            return;
53        }
54        NodeSLL_2511532010 curr_2010 = head_2010;
55        while (curr_2010.next_2010 != null) {
56            System.out.print(curr_2010.data_2010 + "-->");
57            curr_2010 = curr_2010.next_2010;
58        }
59        System.out.println(curr_2010.data_2010);
60    }
61
62    public static void main(String[] args) {
63
64        NodeSLL_2511532010 head_2010 = new NodeSLL_2511532010(1);
65        head_2010.next_2010 = new NodeSLL_2511532010(2);
66        head_2010.next_2010.next_2010 = new NodeSLL_2511532010(3);
67        head_2010.next_2010.next_2010.next_2010 = new NodeSLL_2511532010(4);
68        head_2010.next_2010.next_2010.next_2010.next_2010 = new NodeSLL_2511532010(5);
69        head_2010.next_2010.next_2010.next_2010.next_2010.next_2010 = new NodeSLL_2511532010(6);
70
71        System.out.print("List awal : ");
72        printList(head_2010);
73
74        head_2010 = deteleHead(head_2010);
75        System.out.print("List setelah head dihapus : ");
76        printList(head_2010);
77
78        head_2010 = removeLastNode(head_2010);
79        System.out.print("List setelah simpul terakhir dihapus : ");
80        printList(head_2010);
81
82        int position_2010 = 2;
83        head_2010 = deleteNode(head_2010, position_2010);
84        System.out.print("List setelah posisi 2 dihapus : ");
85        printList(head_2010);
86    }
87 }

```

1. Struktur Kelas HapusSLL_2511532010

Kelas ini dirancang khusus untuk menangani berbagai skenario penghapusan data pada struktur *Singly Linked List*.

2. Logika deleteHead

Fungsi ini menghapus simpul paling depan dengan cara memindahkan penunjuk utama (head) ke simpul berikutnya (head.next), sehingga simpul pertama yang lama terlepas dari daftar.

3. Logika removeLastNode

Fungsi ini menghapus simpul paling belakang. Program akan mencari simpul "kedua terakhir" terlebih dahulu, lalu memutus hubungannya dengan simpul terakhir dengan mengubah pointer next menjadi null.

4. Logika deleteNode

Fungsi ini menghapus simpul berdasarkan nomor urutan yang diinput. Jika yang dihapus posisi di tengah, program akan menyambungkan simpul sebelumnya langsung ke simpul setelah target, sehingga simpul target "dilompati".

5. Validasi Data

Dalam metode deleteNode, terdapat pengecekan; jika posisi yang diminta tidak ditemukan atau melebihi panjang list, program akan menampilkan pesan "Data tidak ada".

6. Fungsi printList

Digunakan untuk menampilkan seluruh isi list ke layar. Fungsi ini menelusuri setiap simpul satu per satu dan mencetak datanya dengan format panah (-->) sebagai visualisasi hubungan antar simpul.

7. Inisialisasi Data di main

Di bagian ini, program membuat sebuah list awal secara manual yang berisi urutan angka dari 1 sampai 6 untuk keperluan pengujian.

8. Simulasi Penghapusan

Metode main menjalankan tiga jenis penghapusan secara berurutan (hapus depan, hapus belakang, dan hapus posisi ke-2) serta mencetak hasilnya setiap kali operasi selesai dilakukan untuk memastikan kode bekerja dengan benar.

Hasil :

List awal : 1-->2-->3-->4-->5-->6
List setelah head dihapus : 2-->3-->4-->5-->6
List setelah simpul terakhir dihapus : 2-->3-->4-->5
List setelah posisi 2 dihapus : 2-->4-->5

2.3 PencarianSLL_2511532010

```
1 package pekan5_2511532010;
2 import java.util.*;
3 public class PencarianSLL_2511532010 {
4     static boolean searchKey(NodeSLL_2511532010 head_2010, int key_2010) {
5         NodeSLL_2511532010 curr_2010 = head_2010;
6         while (curr_2010 != null) {
7             if (curr_2010.data_2010 == key_2010)
8                 return true;
9             curr_2010 = curr_2010.next_2010;
10        }
11        return false;
12    }
13    public static void traversal (NodeSLL_2511532010 head_2010) {
14        NodeSLL_2511532010 curr_2010 = head_2010;
15        while (curr_2010 != null) {
16            System.out.print (" " + curr_2010.data_2010);
17            curr_2010 = curr_2010.next_2010;
18            System.out.println();
19        }
20    }
21    public static void main(String[] args) {
22        NodeSLL_2511532010 head = new NodeSLL_2511532010(14);
23        head.next_2010 = new NodeSLL_2511532010(21);
24        head.next_2010.next_2010 = new NodeSLL_2511532010(13);
25        head.next_2010.next_2010.next_2010 = new NodeSLL_2511532010(30);
26        head.next_2010.next_2010.next_2010.next_2010 = new NodeSLL_2511532010(10);
27        System.out.print("Penelusuran SLL: ");
28        traversal(head);
29        int key = 30;
30        System.out.print("Cari data " + key + " = ");
31
32        if (searchKey(head, key)) {
33            System.out.println("ketemu");
34        } else {
35            System.out.println("tidak ada");
36        }
37    }
38 }
39
40
```

1. Definisi Kelas

Kelas PencarianSLL_2511532010 dibuat untuk mengelola operasi pencarian dan penampilan data pada struktur Singly Linked List.

2. Fungsi searchKey

Metode ini bertugas memindai isi list untuk menemukan keberadaan nilai tertentu yang disebut sebagai key_2010.

3. Proses Iterasi

Menggunakan perulangan while untuk mengunjungi setiap simpul (node) selama simpul tersebut tidak kosong atau bernilai null.

4. Logika Pencocokan

Di dalam setiap langkah, program membandingkan data di simpul saat ini dengan kunci; jika sama, fungsi langsung mengembalikan nilai true.

5. Hasil Akhir Pencarian

Jika seluruh simpul telah diperiksa dan tidak ada data yang cocok, fungsi akan mengembalikan nilai false sebagai tanda data tidak ditemukan.

6. Fungsi traversal

Metode ini berguna untuk mencetak seluruh isi data di dalam list ke layar secara berurutan agar alur data terlihat jelas.

7. Persiapan Data

Pada metode main, program membangun rantai simpul secara manual yang berisi angka 14, 21, 13, 30, dan 10.

8. Target Pencarian

Variabel key diinisialisasi dengan angka 30, yang merupakan nilai yang ingin diuji keberadaannya dalam daftar tersebut.

9. Percabangan Output

Program menggunakan struktur if-else untuk memutuskan pesan mana yang akan dicetak berdasarkan hasil dari fungsi searchKey.

10. Kesimpulan Eksekusi

Karena angka 30 ada di dalam daftar simpul yang dibuat, maka hasil akhir yang muncul di layar adalah pesan "ketemu".

Hasil :

Penelusuran SLL: 14 21 13 30 10
Cari data 30 = ketemu

2.4 TambahSLL_2511532010

```

1 package pekan5_2511532010;
2
3 public class TambahSLL_2511532010 {
4     public static NodeSLL_2511532010 insertAtFront(NodeSLL_2511532010 head_2010, int value_2010) {
5         NodeSLL_2511532010 new_node_2010 = new NodeSLL_2511532010(value_2010);
6         new_node_2010.next_2010 = head_2010;
7         return new_node_2010;
8     }
9
10    public static NodeSLL_2511532010 insertAtEnd(NodeSLL_2511532010 head_2010, int value_2010) {
11        NodeSLL_2511532010 new_node_2010 = new NodeSLL_2511532010(value_2010);
12        if (head_2010 == null) {
13            return new_node_2010;
14        }
15        NodeSLL_2511532010 last_2010 = head_2010;
16        while (last_2010.next_2010 != null) {
17            last_2010 = last_2010.next_2010;
18        }
19        last_2010.next_2010 = new_node_2010;
20        return head_2010;
21    }
22
23    static NodeSLL_2511532010 GetNode(int data) {
24        return new NodeSLL_2511532010(data);
25    }
26
27    static NodeSLL_2511532010 insertPos(NodeSLL_2511532010 headNode, int position, int value_2010) {
28        NodeSLL_2511532010 head_2010 = headNode;
29        if (position < 1) {
30            System.out.println("Invalid position");
31            return head_2010;
32        }
33        if (position == 1) {
34            NodeSLL_2511532010 new_node_2010 = new NodeSLL_2511532010(value_2010);
35            new_node_2010.next_2010 = head_2010;
36            return new_node_2010;
37        } else {
38            // Perbaiki logika traversal agar tidak null pointer
39            while (position-- != 0 && headNode != null) {
40                if (position == 1) {
41                    NodeSLL_2511532010 new_node_2010 = GetNode(value_2010);
42                    new_node_2010.next_2010 = headNode.next_2010;
43                    headNode.next_2010 = new_node_2010;
44                    return head_2010;
45                }
46                headNode = headNode.next_2010;
47            }
48            System.out.println("posisi di luar jangkauan");
49            return head_2010;
50        }
51    } // Penutup method insertPos yang benar
52
53    public static void printList(NodeSLL_2511532010 head_2010) {
54        if (head_2010 == null) {
55            System.out.println("List kosong");
56            return;
57        }
58        NodeSLL_2511532010 curr_2010 = head_2010;
59        while (curr_2010.next_2010 != null) {
60            System.out.print(curr_2010.data_2010 + "-->");
61            curr_2010 = curr_2010.next_2010;
62        }
63        System.out.println(curr_2010.data_2010);
64    }
65
66    public static void main(String[] args) {
67        NodeSLL_2511532010 head_2010 = new NodeSLL_2511532010(2);
68        head_2010.next_2010 = new NodeSLL_2511532010(3);
69        head_2010.next_2010.next_2010 = new NodeSLL_2511532010(5);
70        head_2010.next_2010.next_2010.next_2010 = new NodeSLL_2511532010(6);
71
72        System.out.print("Senarai berantai awal ; ");
73        printList(head_2010);
74        System.out.print("tambah 1 simpul di depan : ");
75        int data_2010 = 1;
76        head_2010 = insertAtFront(head_2010, data_2010);
77        printList(head_2010);
78        System.out.print("tambah 1 simpul di belakang :");
79        int data2_2010 = 7;
80        head_2010 = insertAtEnd(head_2010, data2_2010);
81        printList(head_2010);
82        System.out.print("tambah 1 simpul ke data 4");
83        int data3_2010 = 4;
84        int pos_2010 = 4;
85        head_2010 = insertPos(head_2010, pos_2010, data3_2010);
86        printList(head_2010);
87    }
88 }

```

1. Metode insertAtFront

Digunakan untuk menambah simpul di posisi paling depan dengan cara mengarahkan pointer next simpul baru ke kepala (head) list saat ini.

2. Metode insertAtEnd

Berfungsi menambah simpul di posisi paling belakang dengan menelusuri list hingga ujung, lalu menyambungkan simpul terakhir ke simpul baru.

3. Metode GetNode

Sebuah fungsi pembantu (*helper*) yang bertugas menginisialisasi dan membuat objek node baru dari kelas NodeSLL_2511532010.

4. Metode insertPos

Memungkinkan penyisipan data di posisi manapun (tengah) berdasarkan nomor urut posisi yang dimasukkan oleh pengguna.

5. Logika Pergeseran

Di dalam insertPos, program melakukan iterasi menggunakan while untuk menemukan simpul tepat sebelum posisi yang dituju agar hubungan antar simpul bisa disambungkan ulang.

6. Pengecekan Validitas

Terdapat validasi untuk memastikan posisi yang diinput tidak kurang dari 1 atau tidak melebihi kapasitas list yang tersedia guna menghindari kesalahan sistem.

7. Metode printList

Berfungsi menampilkan seluruh elemen list ke layar secara berurutan dengan menggunakan format tanda panah (-->) sebagai pemisah.

Inisialisasi di Main: Pada metode utama, program pertama kali membuat rantai list awal yang berisi empat angka, yaitu 2, 3, 5, dan 6.

8. Simulasi Penambahan

Program mendemonstrasikan penambahan angka 1 di depan, angka 7 di belakang, dan menyisipkan angka 4 di posisi ke-4, lalu mencetak setiap hasilnya secara bertahap.

```
Senarai berantai awal ; 2-->3-->5-->6  
tambah 1 simpul di depan : 1-->2-->3-->5-->6  
tambah 1 simpul di belakang :1-->2-->3-->5-->6-->7  
tambah 1 simpul ke data 41-->2-->3-->4-->5-->6-->7
```

BAB III

KESIMPULAN

3.1 Ringkasan Hasil Praktikum

Hasil praktikum ini memperjelas bahwa Singly Linked List menawarkan keleluasaan dalam pengelolaan memori yang jauh melampaui kemampuan array statis. Karena setiap simpul (node) saling bertaut melalui sistem referensi, pengisian data tidak lagi bergantung pada ketersediaan blok memori yang berurutan secara fisik. Memahami arsitektur simpul ini menjadi landasan kuat bagi pengembang untuk merancang sistem yang mampu menyesuaikan diri secara otomatis terhadap lonjakan volume data tanpa mengorbankan stabilitas performa aplikasi.

Di sisi lain, penerapan logika operasional dalam menyisipkan, menghapus, dan mencari simpul terbukti efektif mengasah ketelitian dalam memelihara integritas rantai referensi agar terhindar dari galat alamat atau kebocoran memori. Keterampilan memanipulasi elemen list ini bukan hanya sekadar teknis dasar, melainkan fondasi berpikir yang sangat krusial sebelum melangkah ke struktur data yang lebih kompleks seperti Stack, Queue, hingga Tree. Pada akhirnya, praktikum ini berhasil menyatukan pemahaman teori dengan kecakapan praktis dalam mengelola alur data secara rapi dan efisien.

DAFTAR PUSTAKA

- [1] Oracle, *The Java® Tutorial — Primitive Data Types*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>
- [2] GeeksforGeeks, *Java Data Types — GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/data-types-in-java>

TUGAS

1. Pasien_2511532010

```
1 package pekan5_2511532010;
2
3 public class Pasien_2511532010 {
4     private String namaPasien_2010;
5     private String keluhan_2010;
6     private int nomorAntrian_2010;
7     private Pasien_2511532010 next_2010;
8
9     // Constructor
10    public Pasien_2511532010(String namaPasien_2010, String keluhan_2010, int nomorAntrian_2010) {
11        this.namaPasien_2010 = namaPasien_2010;
12        this.keluhan_2010 = keluhan_2010;
13        this.nomorAntrian_2010 = nomorAntrian_2010;
14        this.next_2010 = null;
15    }
16
17    // Selektor (Getter)
18    public String getNamaPasien_2010() { return namaPasien_2010; }
19    public String getKeluhan_2010() { return keluhan_2010; }
20    public int getNomorAntrian_2010() { return nomorAntrian_2010; }
21    public Pasien_2511532010 getNext_2010() { return next_2010; }
22
23    // Mutator (Setter)
24    public void setNamaPasien_2010(String nama_2010) { this.namaPasien_2010 = nama_2010; }
25    public void setKeluhan_2010(String keluhan_2010) { this.keluhan_2010 = keluhan_2010; }
26    public void setNomorAntrian_2010(int nomor_2010) { this.nomorAntrian_2010 = nomor_2010; }
27    public void setNext_2010(Pasien_2511532010 next_2010) { this.next_2010 = next_2010; }
28 }
```

2. RumahSakit_2511532010

```

1 package pekan5_2511532010;
2
3 import java.util.*;
4
5 public class RumahSakit_2511532010 {
6     private static Pasien_2511532010 head_2010 = null;
7     private static Pasien_2511532010 tail_2010 = null;
8     private static int counter_2010 = 0;
9
10    // 1. Daftarkan Pasien (Insert at Tail)
11    public static void daftarkanPasien_2010(String nama_2010, String keluhan_2010) {
12        counter_2010++;
13        Pasien_2511532010 baru_2010 = new Pasien_2511532010(nama_2010, keluhan_2010, counter_2010);
14
15        if (head_2010 == null) {
16            head_2010 = tail_2010 = baru_2010;
17        } else {
18            tail_2010.setNext_2010(baru_2010);
19            tail_2010 = baru_2010;
20        }
21        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_2010);
22    }
23
24    // 2. Panggil Pasien (Delete Head)
25    public static void panggilPasien_2010() {
26        if (head_2010 == null) {
27            System.out.println("Antrian Kosong!");
28            return;
29        }
30        System.out.println("Memanggil Pasien: " + head_2010.getNamaPasien_2010() + " [Antrian: " + head_2010.getNomorAntrian_2010() + ""];
31        head_2010 = head_2010.getNext_2010();
32        if (head_2010 == null) tail_2010 = null;
33    }
34
35    // 3. Tampilkan Antrian (Display)
36    public static void tampilkanAntrian_2010() {
37        if (head_2010 == null) {
38            System.out.println("Antrian Kosong!");
39            return;
40        }
41        Pasien_2511532010 curr_2010 = head_2010;
42        System.out.println("== Daftar Antrian ==");
43        while (curr_2010 != null) {
44            System.out.println("No: " + curr_2010.getNomorAntrian_2010() + " | Nama: " + curr_2010.getNamaPasien_2010() + " | Keluhan: " + curr_2010.getKeluhan_2010());
45            curr_2010 = curr_2010.getNext_2010();
46        }
47    }
48
49    // 4. Cari Pasien (Search - Case Insensitive)
50    public static void cariPasien_2010(String cariNama_2010) {
51        Pasien_2511532010 curr_2010 = head_2010;
52        boolean ketemu_2010 = false;
53        while (curr_2010 != null) {
54            if (curr_2010.getNamaPasien_2010().equalsIgnoreCase(cariNama_2010)) {
55                System.out.println("Pasien ditemukan! No Antrian: " + curr_2010.getNomorAntrian_2010() + ", Keluhan: " + curr_2010.getKeluhan_2010());
56                ketemu_2010 = true;
57                break;
58            }
59            curr_2010 = curr_2010.getNext_2010();
60        }
61        if (!ketemu_2010) System.out.println("Pasien dengan nama '" + cariNama_2010 + "' tidak ditemukan.");
62    }
63
64    // 5. Cek Status Antrian
65    public static void cekStatusAntrian_2010() {
66        if (head_2010 == null) {
67            System.out.println("List Kosong.");
68            return;
69        }
70        int total_2010 = 0;
71        Pasien_2511532010 curr_2010 = head_2010;
72        while (curr_2010 != null) {
73            total_2010++;
74            curr_2010 = curr_2010.getNext_2010();
75        }
76        System.out.println("Total Pasien dalam Antrian: " + total_2010);
77        System.out.println("Pasien Terdepan: " + head_2010.getNamaPasien_2010() + " (No: " + head_2010.getNomorAntrian_2010() + ")");
78    }
79 }

```

```
public static void main(String[] args) {
    Scanner sc_2010 = new Scanner(System.in);
    int pilihan_2010;

    do {
        System.out.println("\n== Antrian Rumah Sakit NIM: 2511532010 ==");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_2010 = sc_2010.nextInt();
        sc_2010.nextLine(); // consume newline

        switch (pilihan_2010) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String n_2010 = sc_2010.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String k_2010 = sc_2010.nextLine();
                daftarkanPasien_2010(n_2010, k_2010);
                break;
            case 2:
                panggilPasien_2010();
                break;
            case 3:
                tampilkanAntrian_2010();
                break;
            case 4:
                System.out.print("Masukkan Nama Pasien yang dicari: ");
                cariPasien_2010(sc_2010.nextLine());
                break;
            case 5:
                cekStatusAntrian_2010();
                break;
            case 6:
                System.out.println("Terima kasih!");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
        }
    } while (pilihan_2010 != 6);
    sc_2010.close();
}
```

Writable	Smart Insert	6 : 54 [6]	...	💡
----------	--------------	------------	-----	---

HASIL :

```

=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: habib
Masukkan Keluhan: demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: yb
Masukkan Keluhan: pusing
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: gaga
Masukkan Keluhan: muntah
Pasien berhasil didaftarkan! Nomor Antrian: 3

=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien: habib [Antrian: 1]

=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien: yb [Antrian: 2]

```

```
=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== Daftar Antrian ===
No: 3 | Nama: gaga | Keluhan: muntah

=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari: gaga
Pasien ditemukan! No Antrian: 3, Keluhan: muntah

=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Total Pasien dalam Antrian: 1
Pasien Terdepan: gaga (No: 3)

=== Antrian Rumah Sakit NIM: 2511532010 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan:
```

Penjelasan

1. Struktur Linked List

Kode ini mengimplementasikan konsep *Single Linked List* untuk mengelola antrian. Objek `head_2010` berfungsi sebagai penunjuk pasien terdepan, sedangkan `tail_2010` menunjuk ke pasien yang terakhir kali mendaftar.

2. Pendaftaran Pasien (Insert at Tail)

Saat fungsi `daftarPasien_2010` dipanggil, program membuat objek baru. Jika antrian kosong, objek tersebut menjadi head sekaligus tail. Jika sudah ada isinya, objek baru akan disambungkan setelah tail lama, kemudian posisi tail diperbarui ke objek baru tersebut.

3. Pemanggilan Pasien (Delete Head)

Fungsi `panggilPasien_2010` menerapkan prinsip FIFO (*First In First Out*). Program akan mengambil data dari head, lalu menggeser posisi head ke node berikutnya (`getNext_2010`). Jika setelah digeser antrian menjadi kosong, maka tail juga diatur menjadi null.

4. Penelusuran Antrian (Display)

Dalam fungsi `tampilkanAntrian_2010`, program menggunakan variabel bantuan `curr_2010` untuk melakukan *traversal* mulai dari head hingga node terakhir. Selama `curr_2010` tidak bernilai null, data pasien akan dicetak ke layar satu per satu.

5. Pencarian Data (Search)

Fungsi `cariPasien_2010` melakukan perbandingan nama menggunakan `equalsIgnoreCase`. Logika ini memungkinkan pencarian tetap berhasil meskipun ada perbedaan penggunaan huruf kapital pada input user dengan data yang tersimpan di memori.

6. Pemantauan Status (Cek Status)

Fungsi `cekStatusAntrian_2010` menghitung total elemen dalam list secara dinamis dengan melakukan *looping*. Selain jumlah total, fungsi ini juga memberikan

informasi cepat mengenai siapa pasien yang berada di urutan paling depan untuk dipanggil.

7. Manajemen Input dan Menu

Pada metode main, penggunaan `sc_2010.nextLine()` setelah `sc_2010.nextInt()` sangat krusial. Hal ini bertujuan untuk membersihkan karakter *newline* yang tertinggal di buffer agar input nama dan keluhan pada iterasi berikutnya tidak terlewat secara otomatis.